

Parallel and Distributed Systems

Chapter 3: Process-Based Parallelism

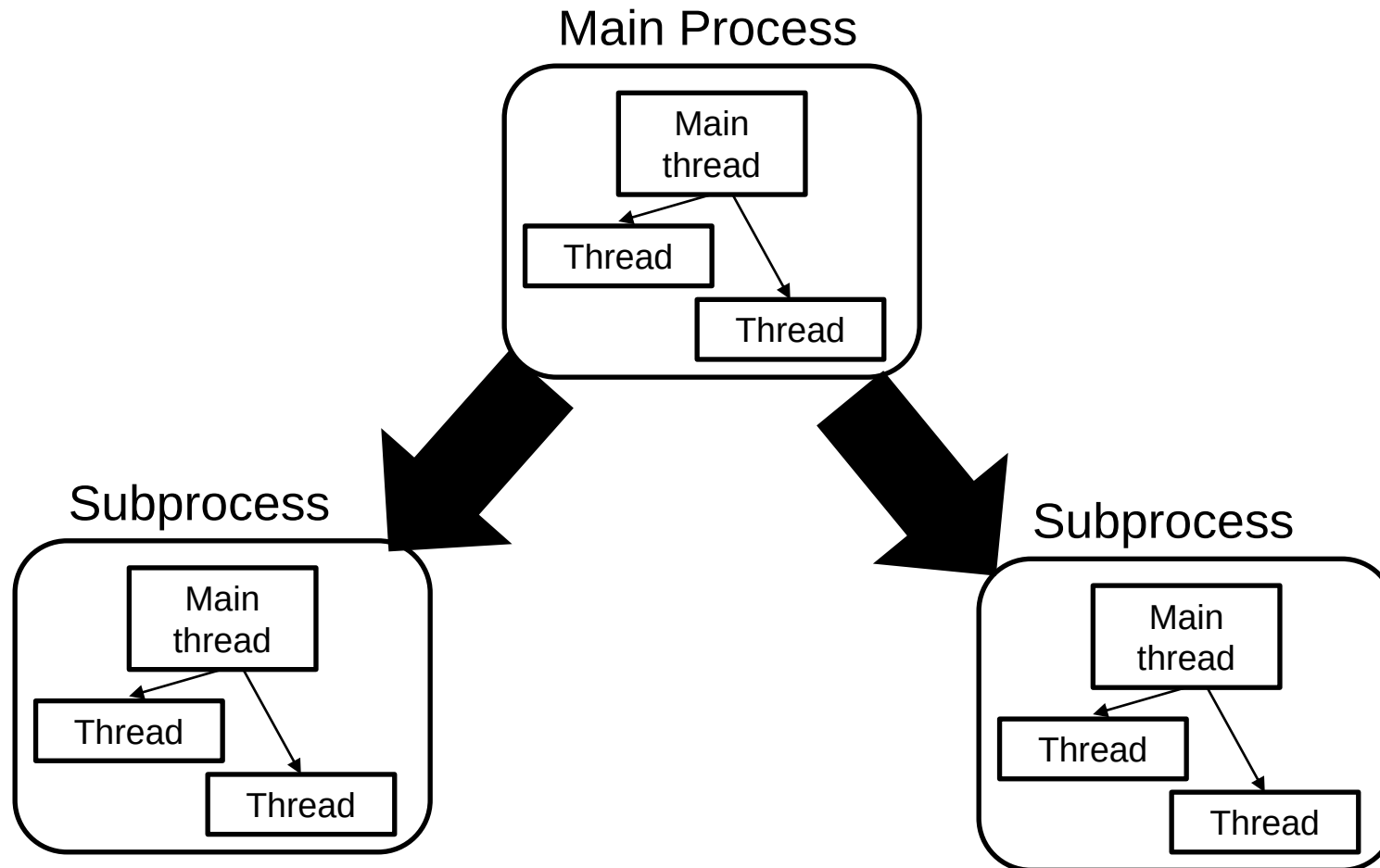
Prof. Dr. Martin Sträßer



Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Structure of Concurrent Programs





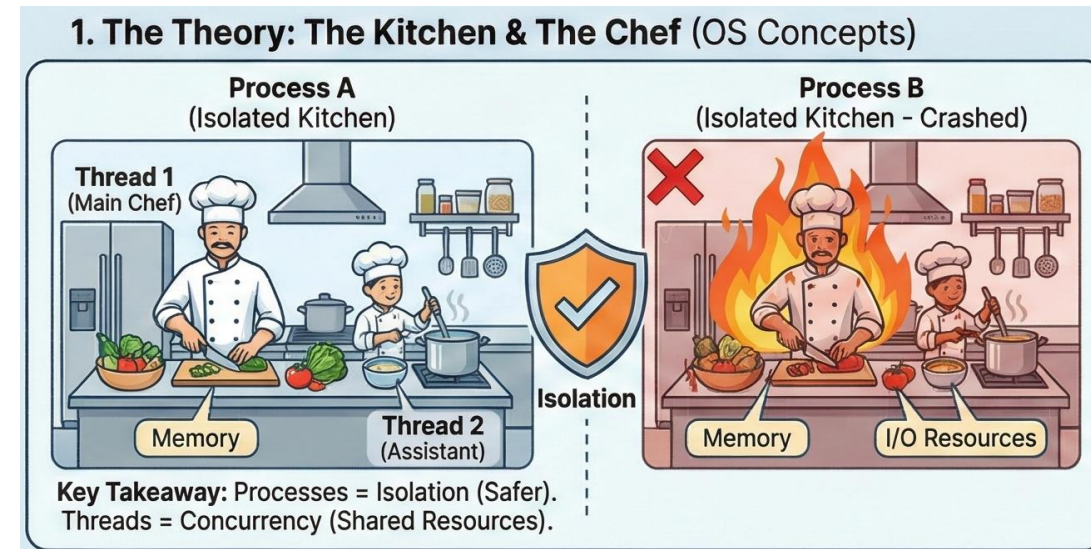
Overview

- Processes from an OS-level perspective
- Processes in Python

Processes from an OS-level perspective

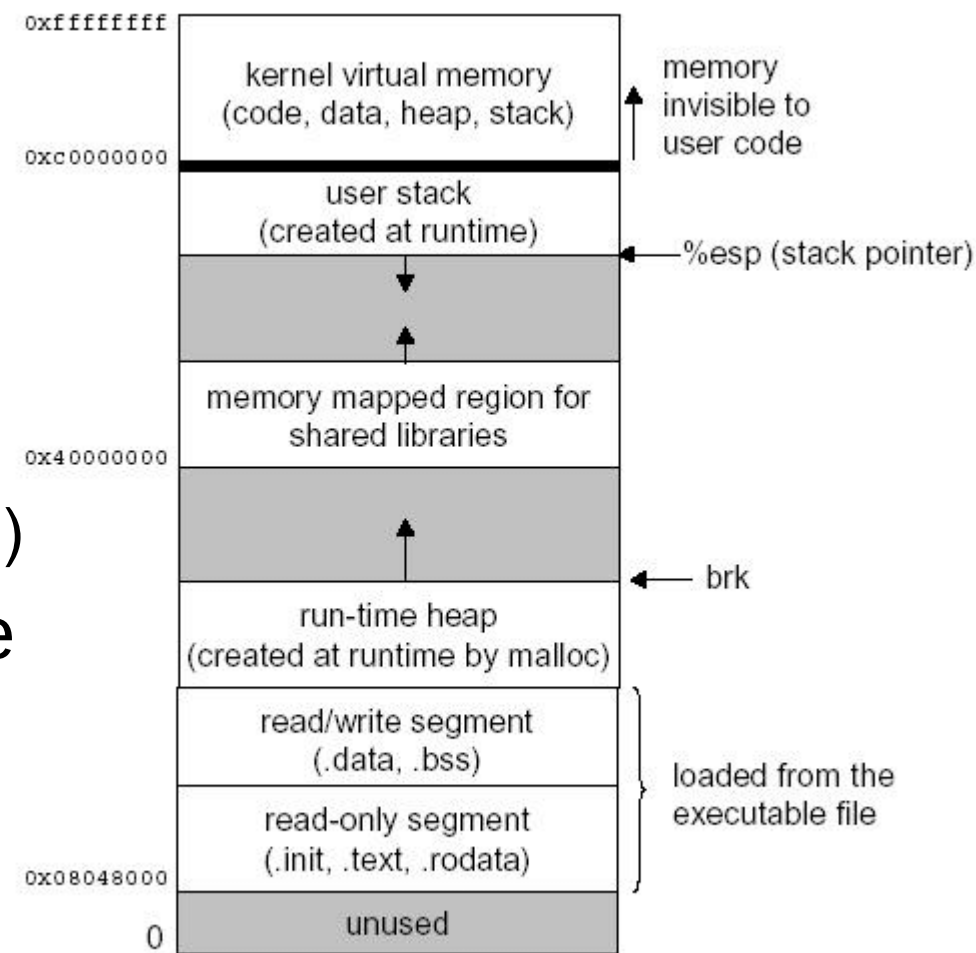
Process

- Entity that encapsulates an executing application
- Spawned and controlled by the operating system
- Process vs. program
 - A program is a passive file on disk
 - A process is an active execution of a program
 - Multiple processes can run the same program at the same time
- Two (unrelated) processes are fully isolated from each other



Process Components

- Virtual address space is the memory region where the process has access to, it contains:
 - Program code
 - Stack and heap
 - Environment/Global variables
 - Memory mappings (shared libraries, files, ...)
- There is also a kernel-level process state independent of the process memory, it contains:
 - Current working directory
 - Kernel metadata (PID, scheduling info, ...)

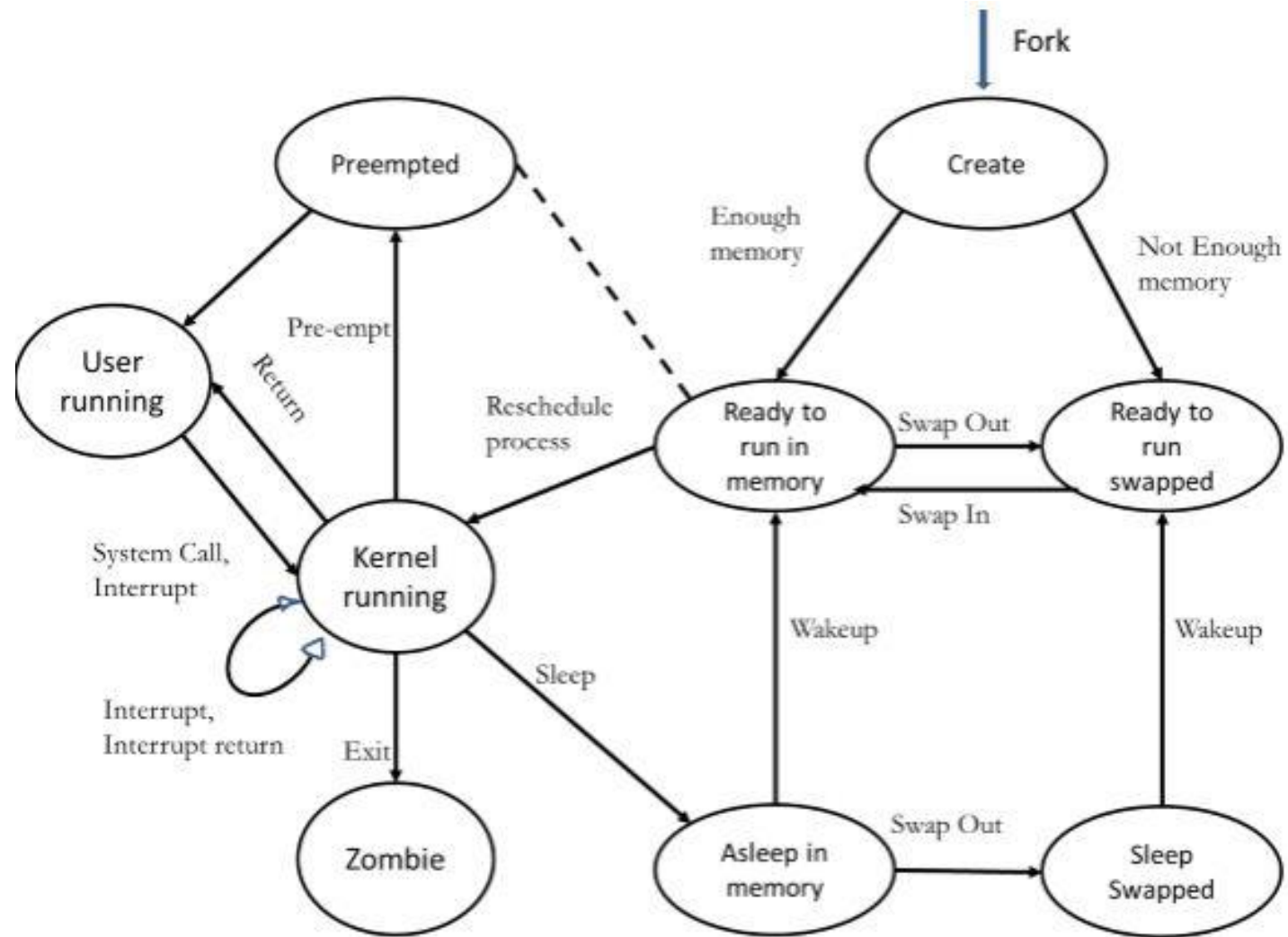




Process States

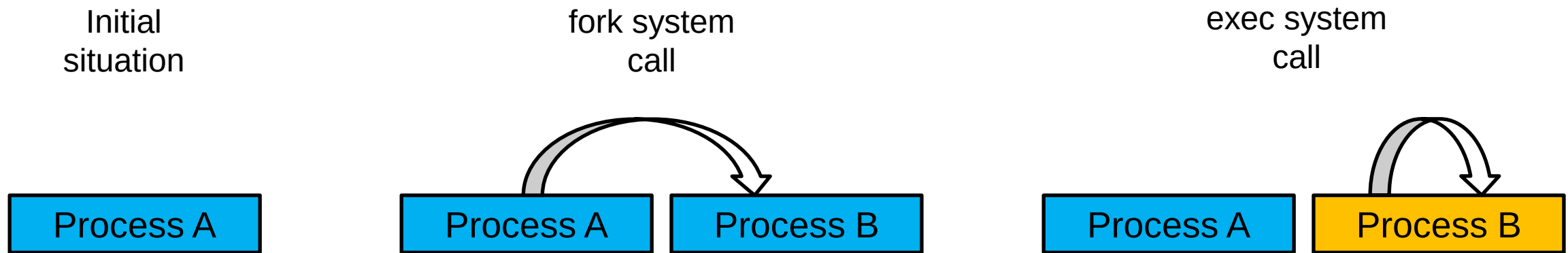
- Question: Which process states do you remember from the operating systems course? What can cause state transitions?

Process States



Process Creation

- Different operating systems provide different mechanisms to create new processes
- Unix/Linux: Two steps consisting of fork and exec



- fork system call creates a clone of the original process
- exec system call replaces the process image (e.g., code to be executed)

Linux Process Creation in Detail

- When a process calls fork:
 - The kernel creates a new process (child)
 - The child is an almost-exact copy of the parent
 - Execution continues in both processes
- Relationship between parent and child after fork

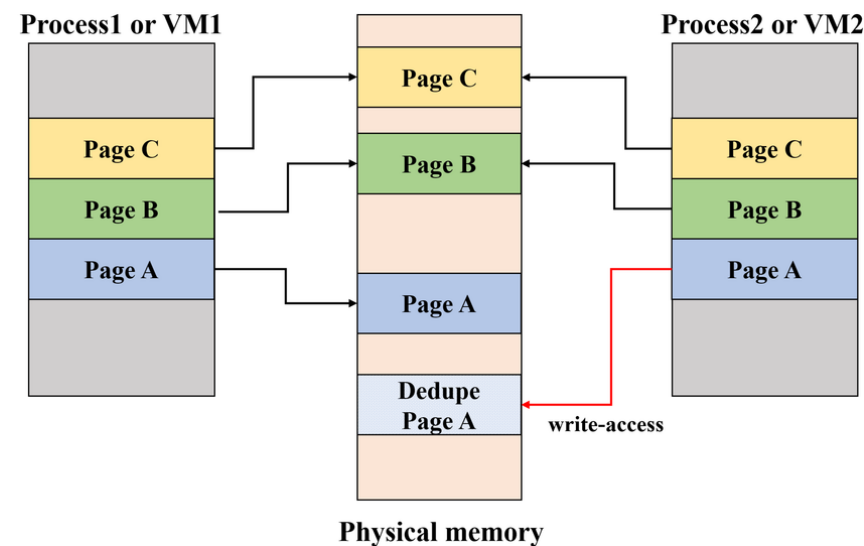
Process Component	Relationship after fork
Program code	Shared
Stack and heap	Shared until first write (copy-on-write)
Environment variables	Shared until first write (copy-on-write)
Open file descriptors	Shared
Signal handlers	Copied
Current working directory	Copied
Kernel metadata	Independent

```
c

pid_t pid = fork();
if (pid == 0) {
    // child
} else {
    // parent
}
```

Linux Process Creation in Detail

- Copy-on-write (COW) is an important mechanism when dealing with shared memory
- Broad usage, not only for Linux process creation (e.g., for Docker containers)
- First, parent and child share the same memory pages
- Once either of them wants to write something to the memory, the writer gets an independent copy of the memory pages
- Question: What is the advantage here?



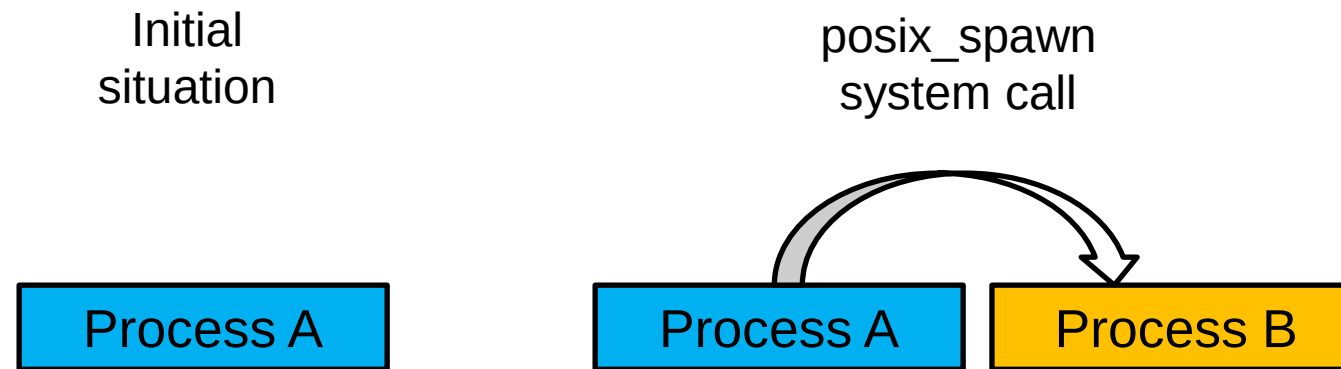
Linux Process Creation in Detail

- After forking, the newly created process typically calls `exec()` to replace its process image
- Effect of `exec()` to process components

Process Component	Relationship after fork	Relationship after exec
Program code	Shared	Independent
Stack and heap	Shared until first write (copy-on-write)	Independent
Environment variables	Shared until first write (copy-on-write)	Programmer's choice
Open file descriptors	Shared	Programmer's choice
Signal handlers	Copied	Independent
Current working directory	Copied	Same as after fork
Kernel metadata	Independent values	Same as after fork

Process Creation

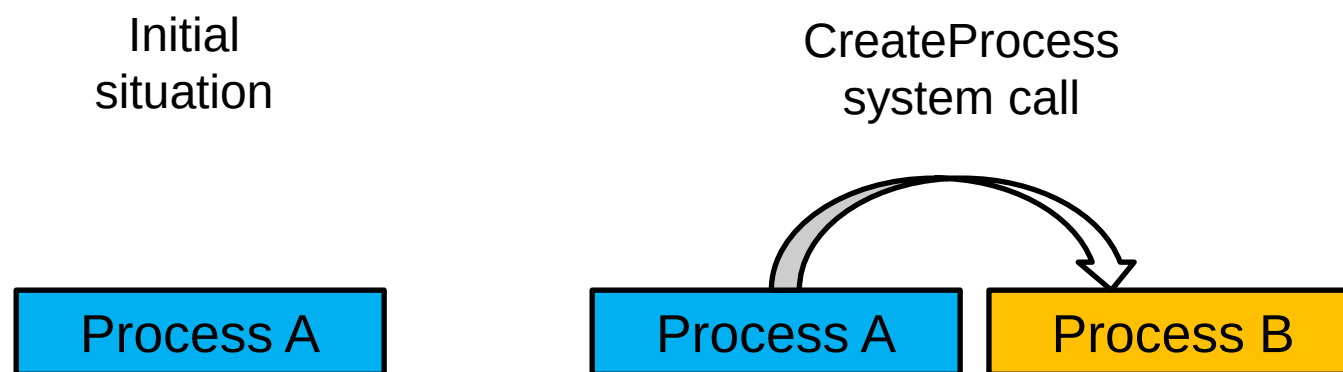
- Different operating systems provide different mechanisms to create new processes
- POSIX/macOS: One step spawning



- `posix_spawn` is a combination of `fork` and `exec`
- Also supported for newer Linux versions
- iOS: Process creation only allowed for system processes, not apps

Process Creation

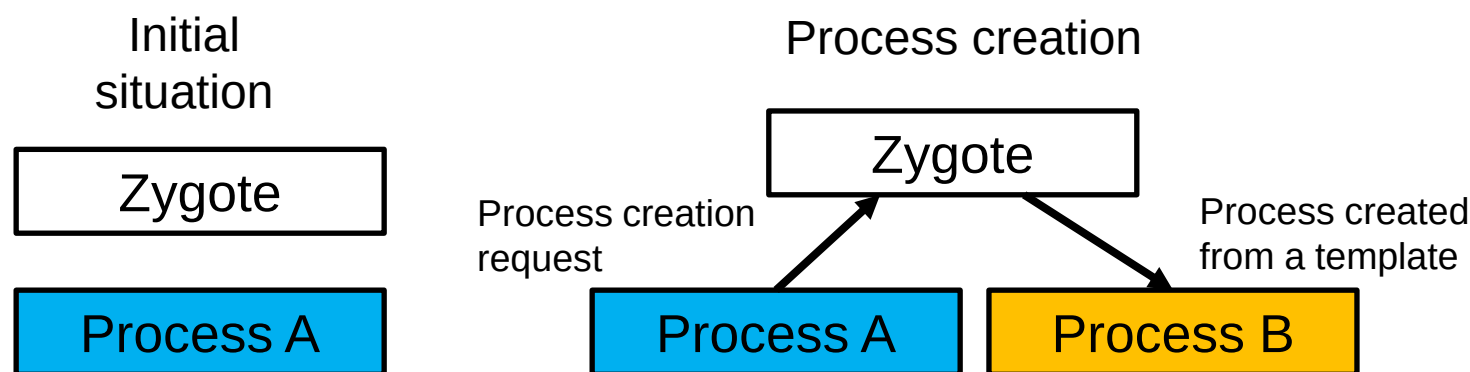
- Different operating systems provide different mechanisms to create new processes
- Windows: One step



- Windows does not support forking/cloning of processes
- Similar to POSIX, creation and code replacement consolidated in one system call

Process Creation

- Different operating systems provide different mechanisms to create new processes
- Android: Managed process creation via Zygote



- Zygote is a system process acting as a process factory
- Advantage: Android processes all require similar libraries and runtimes, Zygote packs into a template allowing for faster startups



Process Termination

- Question: When/why does a process terminate?



Process Termination

- Voluntary termination
 - Process finishes successfully/self-controlled
 - OS-level view: Explicit exit system call is executed
 - In high-level languages: main method/function completed
- Errors or exceptions
 - Handling depends on operating system
 - Unix/Linux: Special signals are sent
 - Windows: Structured exception handling
- Forced external termination
 - Unix/Linux: kill signal
- Resource exhaustion
 - Special cases of forced termination, e.g., Linux OOM Killer

OS-Level Process Management

```
snorlax212@snorlax212: ~/Desktop/lab6
snorlax212@snorlax212:~/Desktop/lab6$ ps -el
F S  UID  PID  PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0    1    0  0  80   0  -  6144 poll_s ?    00:00:00 init
1 S   0    2    0  0  80   0  -    0 kthrea ?    00:00:00 kthreadd
1 S   0    3    2  0  80   0  -    0 smpboo ?    00:00:02 ksoftirqd/0
1 S   0    5    2  0  60 -20  -    0 worker ?    00:00:00 kworker/0:0H
1 S   0    7    2  0  60 -20  -    0 worker ?    00:00:00 kworker/u:0H
1 S   0    8    2  0 -40  -    0 cpu_st ?    00:00:00 migration/0
1 S   0    9    2  0  80   0  -    0 rcu_gp ?    00:00:00 rcu_bh
1 S   0   10    2  0  80   0  -    0 rcu_gp ?    00:00:10 rcu_sched
5 S   0   11    2  0 -40  -    0 smpboo ?    00:00:00 watchdog/0
5 S   0   12    2  0 -40  -    0 smpboo ?    00:00:00 watchdog/1
1 S   0   13    2  0  80   0  -    0 smpboo ?    00:00:02 ksoftirqd/1
1 S   0   14    2  0 -40  -    0 cpu_st ?    00:00:00 migration/1
1 S   0   16    2  0  60 -20  -    0 worker ?    00:00:00 kworker/1:0H
5 S   0   17    2  0 -40  -    0 smpboo ?    00:00:00 watchdog/2
1 S   0   18    2  0  80   0  -    0 smpboo ?    00:00:02 ksoftirqd/2
1 S   0   19    2  0 -40  -    0 cpu_st ?    00:00:00 migration/2
1 S   0   21    2  0  60 -20  -    0 worker ?    00:00:00 kworker/2:0H
5 S   0   22    2  0 -40  -    0 smpboo ?    00:00:00 watchdog/3
1 S   0   23    2  0  80   0  -    0 smpboo ?    00:00:01 ksoftirqd/3
1 S   0   24    2  0 -40  -    0 cpu_st ?    00:00:00 migration/3
1 S   0   26    2  0  60 -20  -    0 worker ?    00:00:00 kworker/3:0H
1 S   0   27    2  0  60 -20  -    0 rescue ?    00:00:00 cpuset
```

Prozesse

Neuen Task ausführen

Task beenden

Name	Status	7% CPU	63% Arbeitss...	1% Datenträ...	0% Netzwerk
Apps (6)					
> Firefox (58)		0%	1.776,8 MB	0 MB/s	0,1 MBit/s
> Microsoft Edge (12)		0%	179,1 MB	0 MB/s	0 MBit/s
> Microsoft Excel		0%	14,6 MB	0 MB/s	0 MBit/s
> Microsoft PowerPoint		0%	259,8 MB	0 MB/s	0 MBit/s
> Task-Manager		0,7%	84,8 MB	0,1 MB/s	0 MBit/s
> Windows-Explorer		0,2%	127,8 MB	0,1 MB/s	0 MBit/s
Hintergrundprozesse (90)					
> Application Frame Host		0%	0,9 MB	0 MB/s	0 MBit/s
> Bildschirmtastatur und Schreib...		0%	4,6 MB	0 MB/s	0 MBit/s
> COM Surrogate		0%	0,6 MB	0 MB/s	0 MBit/s
> COM Surrogate		0%	0,2 MB	0 MB/s	0 MBit/s
> CTF-Ladeprogramm		0%	4,4 MB	0 MB/s	0 MBit/s
> DAX API (2)		0%	2,2 MB	0 MB/s	0 MBit/s
> Device Association Framework...		0%	0,1 MB	0 MB/s	0 MBit/s
> Einstellungen		0%	0 MB	0 MB/s	0 MBit/s
> Elan TrackPoint Service		0%	0,1 MB	0 MB/s	0 MBit/s
> EPD Control Center		0%	0,2 MB	0 MB/s	0 MBit/s

Processes in Python



Python Processes

- Each process has one instance of the Python interpreter that executes Python code
- Python processes are directly mapped to OS-level processes
- Two or more Python processes run concurrently and they can be executed in parallel
- Library for implementation: multiprocessing

Run a Function

```
import time
from multiprocessing import Process

def task():
    time.sleep(2)
    print('Child finished')

if __name__ == '__main__':
    # create a child process
    process = Process(target=task)
    # run the child process
    process.start()
    # wait for the child process to finish
    print('Parent waits for child process...')
    process.join()
    print('Parent finished')
```

See code no. 1



Run a Function with Arguments

```
import time
from multiprocessing import Process

def task(seconds: int, message: str) -> None: 1 usage
    time.sleep(seconds)
    print(message)

if __name__ == '__main__':
    # create the child process
    process = Process(target=task, args=(2, 'Hello, I am the child process'))
    # run the child process
    process.start()
    # wait for the child process to finish
    print('Waiting for the child process...')
    process.join()
```

See code no. 2



Process as a Subclass

```
from time import sleep
from multiprocessing import Process
from multiprocessing import Value

class MyProcess(Process):
    # override the constructor
    def __init__(self, x: int, y: int):
        # execute the base constructor
        Process.__init__(self)
        # store instance variables
        self.x = x
        self.y = y
        # initialize integer attribute (i = integer)
        self.data = Value('i', 0)

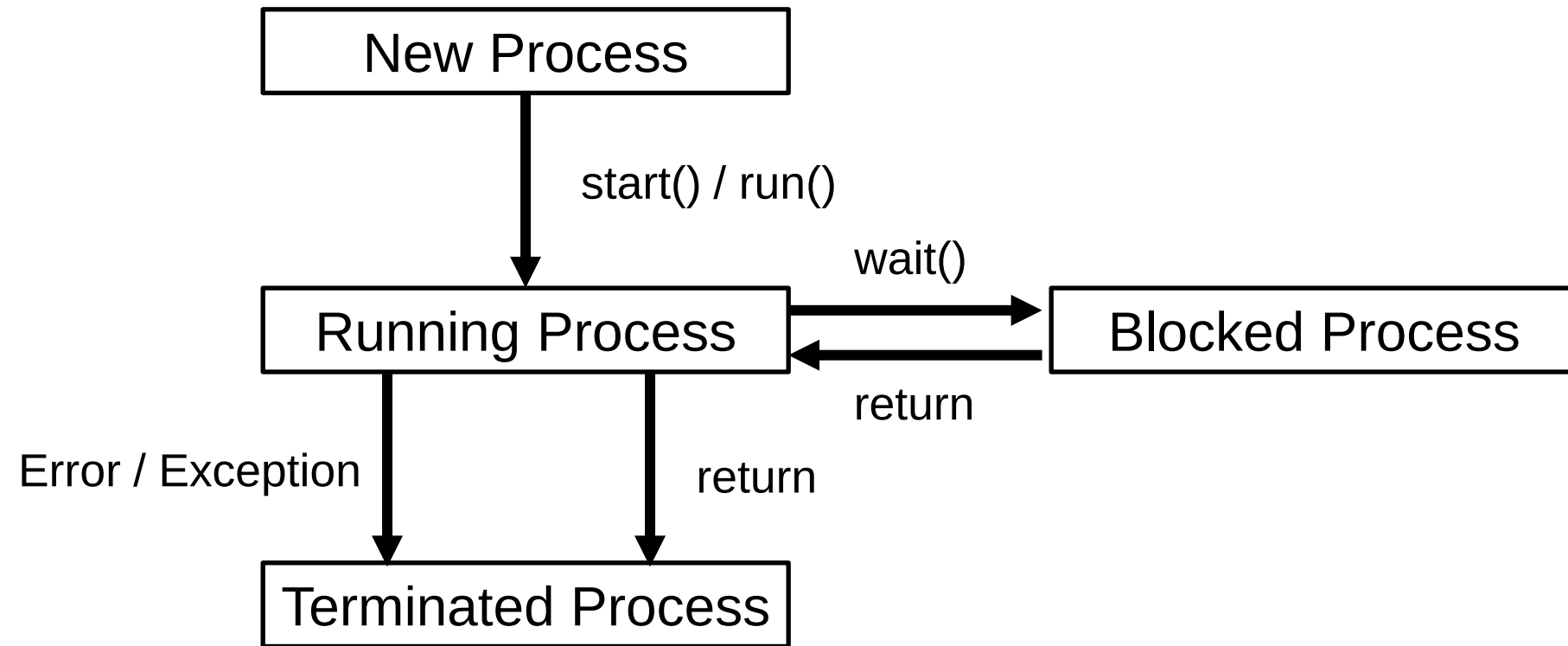
    def run(self):
        sleep(2)
        # store the data variable
        self.data.value = self.x + self.y
        # report stored value
        print(f'Child stored: {self.data.value}')
```

```
if __name__ == '__main__':
    process = MyProcess(x=2, y=4)
    process.start()
    print('Waiting for the child process to finish')
    # wait until child process is terminated
    process.join()
    # report the process attribute
    print(f'Parent got: {process.data.value}')
```

See code no. 3+4



Lifecycle of Python Processes



What Happens Here?

Processes cannot be restarted!
They are single-use objects.

```
Parent starts child
Parent waits for child
Child started
Child finished
Parent ends waiting
Parent restarts child
Traceback (most recent call last):
  File "C:\Users\e01530\PycharmProjects\PVSChapter3\main.py", line 17, in <module>
    process.start()
    ~~~~~^
  File "C:\Users\e01530\AppData\Local\Programs\Python\Python314\Lib\multiprocessing\process.py", line 115, in start
    assert self._popen is None, 'cannot start a process twice'
           ^^^^^^^^^^^^^^^^^
AssertionError: cannot start a process twice

Process finished with exit code 1
```

See code no. 5

```
from multiprocessing import Process
import time

def task():
    print("Child started")
    time.sleep(5)
    print("Child finished")

if __name__ == '__main__':
    process = Process(target=task)
    print("Parent starts child")
    process.start()
    print("Parent waits for child")
    process.join()
    print("Parent ends waiting")
    print("Parent restarts child")
    process.start()
    print("Parent waits for child")
    process.join()
    print("Parent ends waiting")
```

Relationship Between Parent and Child Process

- Typically, a parent process spawns a child process and waits for its return using the `join()` method
- But, if the parent does not wait for the child, what happens to the child once the parent terminated?
- **Nothing, the child keeps running. Child processes are independent OS processes. They become orphans.**

```
from multiprocessing import Process
import time

def task():
    print("Child starts")
    time.sleep(10)
    print("Child finished")

if __name__ == '__main__':
    print("Parent starts")
    process = Process(target=task)
    process.start()
    print("Parent finished")
```

See code no. 6



Daemon Processes

- Daemon processes (also called background processes) terminate immediately if the main (parent) process terminates

```
from multiprocessing import Process
import time

def task():
    print("Child starts")
    time.sleep(10)
    print("Child finished")

if __name__ == '__main__':
    print("Parent starts")
    process = Process(target=task, daemon=True)
    process.start()
    print("Parent finished")
```

See code no. 6



Query Process Attributes

- What will be the output here?

See code no. 7

```
from multiprocessing import Process
import time

def printProcessDetails(process: Process): 2 usages
    print(f'Process name: {process.name}')
    print(f'Process ID: {process.pid}')
    print(f'Is daemon process: {process.daemon}')
    print(f'Process exitcode: {process.exitcode}')
    print(f'Is process alive: {process.is_alive()}')

def task(): 1 usage
    time.sleep(5)
    print('Child finished')

if __name__ == '__main__':
    process = Process(target=task)
    process.start()
    printProcessDetails(process)
    process.join()
    printProcessDetails(process)
```



Identify the Main Process

```
import multiprocessing
import time

def task():
    time.sleep(5)
    me = multiprocessing.current_process()
    print(f'Child name: {me.name}')
    print(f'Child pid: {me.pid}')

if __name__ == '__main__':
    process = multiprocessing.Process(target=task)
    process.start()
    mainProcess = multiprocessing.current_process()
    print(f'main process name: {mainProcess.name}')
    print(f'main process pid: {mainProcess.pid}')
    process.join()
```

See code no. 8



Querying the Parent Process

- A toxic relationship – what happens here?

See code no. 9

```
import time
from multiprocessing import Process
from multiprocessing.process import parent_process

def task():
    time.sleep(2)
    parent = parent_process()
    print(parent.name)
    parent.kill()
    print("Child killed parent")
    time.sleep(2)
    print("Child finished")

if __name__ == '__main__':
    process = Process(target=task)
    process.start()
    print("Parent waiting for child")
    process.join()
    print("Parent finished")
```



Querying the Parent Process

- A toxic relationship – what happens here?
- Solution:
 - The child process will end with an exception
 - The `parent_process` function returns a proxy process object that contains the correct metadata, but we have no control over the parent process

```
import time
from multiprocessing import Process
from multiprocessing.process import parent_process

def task():
    1 usage
    time.sleep(2)
    parent = parent_process()
    print(parent.name)
    parent.kill()
    print("Child killed parent")
    time.sleep(2)
    print("Child finished")

if __name__ == '__main__':
    process = Process(target=task)
    process.start()
    print("Parent waiting for child")
    process.join()
    print("Parent finished")
```


Killing a Process

- In contrast, a parent process is allowed to kill a child process with the `kill()` method

```
import time
from multiprocessing import Process

def task(): 1 usage
    time.sleep(5)
    print("Child finished")

if __name__ == '__main__':
    process = Process(target=task)
    process.start()
    time.sleep(2)
    process.kill()
    print("Parent killed child")
    process.join()
    print("Parent finished")
```

See code no. 10

Other Useful Functions

- `multiprocessing.active_children()`
 - returns a list of all processes that are children of the current process
- `multiprocessing.cpu_count()` – returns amount of CPU cores present in current system

